

Sandboxing gegen Supply-Chain-Attacken



Michael Zugelder

2026-05-07

Überblick

1. Bestandsaufnahme
2. Sandboxing mit Demos
3. ... und noch viel weiter!

Bestandsaufnahme

Supply-Chain-Security

CC BY-NC 2.5 <https://xkcd.com/1200/>
<https://editor.p5js.org/isohedral/full/vja5RiZWs>

Wie schlimm ist es?

Wie schlimm ist es?

- PyPI verteilt Malware (z.B. PyTorch-nightly, PyTorch Lightning, LiteLLM, num2words, elementary-data)

Wie schlimm ist es?

- PyPI verteilt Malware (z.B. PyTorch-nightly, PyTorch Lightning, LiteLLM, num2words, elementary-data)
- Maven(Central) verteilt Malware
(org.fasterxml.jackson.core:jackson-databind,
com.github.codingandcoding:maven-compiler-plugin)

Wie schlimm ist es?

- PyPI verteilt Malware (z.B. PyTorch-nightly, PyTorch Lightning, LiteLLM, num2words, elementary-data)
- Maven(Central) verteilt Malware (org.fasterxml.jackson.core:jackson-databind, com.github.codingandcoding:maven-compiler-plugin)
- Microsoft verteilt Malware (npm):
 - Juli 2025: eslint-config-prettier, got-fetch, ...
 - August 2025: nx
 - September 2025: chalk, debug, ansi-styles, ...
 - November 2025: posthog, postman, @asynccapi, ...
 - März 2026: axios
 - April 2026: @cap-js (SAP), Bitwarden CLI, ...

Wie schlimm ist es?

- PyPI verteilt Malware (z.B. PyTorch-nightly, PyTorch Lightning, LiteLLM, num2words, elementary-data)
- Maven(Central) verteilt Malware (org.fasterxml.jackson.core:jackson-databind, com.github.codingandcoding:maven-compiler-plugin)
- Microsoft verteilt Malware (npm):
 - Juli 2025: eslint-config-prettier, got-fetch, ...
 - August 2025: nx
 - September 2025: chalk, debug, ansi-styles, ...
 - November 2025: posthog, postman, @asynccapi, ...
 - März 2026: axios
 - April 2026: @cap-js (SAP), Bitwarden CLI, ...
- Debian (unstable) verteilt Malware (xz)

Schutzziele

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen?
(SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen? (SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)
- Schutz vor organisierter Kriminalität? (Ransomware, ...)

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen? (SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)
- Schutz vor organisierter Kriminalität? (Ransomware, ...)
- Schutz vor Geheimdiensten?

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfliegen?
- Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen? (SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)
- Schutz vor organisierter Kriminalität? (Ransomware, ...)
- Schutz vor Geheimdiensten?
- Schutz vor US-Firmen/Behörden? (FISA section 702, CLOUD act, National security letters, ...)

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen? (SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)
- Schutz vor organisierter Kriminalität? (Ransomware, ...)
- ~~Schutz vor Geheimdiensten?~~
- ~~Schutz vor US-Firmen/Behörden? (FISA section 702, CLOUD act, National security letters, ...)~~

Schutzziele

- Schutz vor 13-Jährigen, die F12 drücken können?
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- ~~Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen?
(SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)~~
- Schutz vor organisierter Kriminalität? (Ransomware, ...)
- ~~Schutz vor Geheimdiensten?~~
- ~~Schutz vor US-Firmen/Behörden? (FISA section 702, CLOUD act, National security letters, ...)~~

Schutzziele

- ~~Schutz vor 13-Jährigen, die F12 drücken können?~~
- Schutz davor, dass hunderte/tausende unbezahlte Freiwillige *niemals* auf eine Phishing-Mail hereinfallen?
- ~~Schutz vor "Security"-Produkten, die oft mit viel zu hohen Privilegien laufen und peinlichste Schwachstellen aufreißen?
(SolarWinds, Ivanti, Cisco, F5, SonicWall, TrendMicro, Microsoft Malware Protection Engine, ...)~~
- Schutz vor organisierter Kriminalität? (Ransomware, ...)
- ~~Schutz vor Geheimdiensten?~~
- ~~Schutz vor US-Firmen/Behörden? (FISA section 702, CLOUD act, National security letters, ...)~~

Gängige Gegenmaßnahmen

- Dependency-Pinning
- Pakete nicht schon Code allein beim Installieren ausführen lassen
- Updates nicht sofort machen

Gegenmaßnahmen

Dependency-Pinning

Gegenmaßnahmen

Dependency-Pinning

- Grundidee: Abhängigkeiten exakt im Repository definieren
- **Absolut sinnvoll und notwendige Voraussetzung.**
- Aber: kann Updates verlangsamen (Tools wie **Renovate** helfen)
- Teufel im Detail:
 - GitHub-Actions und Docker-Images müssen über Digest gepinnt werden
 - Manche Entwicklungs-Tools installieren trotzdem neueste Versionen

Korrektes Dependency-Pinning

Korrektes Dependency-Pinning

Docker

- `docker.io/node` ❌
- `docker.io/node:latest` ❌
- `docker.io/node:24.15` ❌
- `docker.io/node:24.15.0` ❌
- `docker.io/node:24.15.0@sha256:3a198147c320bc4...` ✅

Korrektes Dependency-Pinning

GitHub Actions

- `actions/checkout` ❌
- `actions/checkout@main` ❌
- `actions/checkout@v5` ❌
- `actions/checkout@v5.0.0` ❌
- `actions/checkout@08c6903cd8c0fde910a... # v5.0.0` ✅

Korrektes Dependency-Pinning

Entwicklungstools, die ungefragt brandneue Versionen installieren

- Nx Console (VS Code Extension)
 - hatte explizit **`npx nx@latest --version`** ausgeführt (und damit alleine durchs IDE starten Malware installiert)
 - hat ein AI-Check Feature (Stand April 2026), was über Timer vollautomatisch die Malware in **`axios`** installiert hat
- Gibt vermutlich viele Beispiele, aber meist passiert ja nichts...

Gegenmaßnahmen

Code nicht während der Installation ausführen

Gegenmaßnahmen

Code nicht während der Installation ausführen

- Keine schlechte Idee, aber bei **npm** mit **ignore-scripts**:

Gegenmaßnahmen

Code nicht während der Installation ausführen

- Keine schlechte Idee, aber bei **npm** mit **ignore-scripts**:
- Deaktiviert auch die *eigenen* Scripts

Gegenmaßnahmen

Code nicht während der Installation ausführen

- Keine schlechte Idee, aber bei **npm** mit **ignore-scripts**:
- Deaktiviert auch die *eigenen* Scripts
- Malware braucht **postinstall** nicht, um Schaden anzurichten

Gegenmaßnahmen

Code nicht während der Installation ausführen

- Keine schlechte Idee, aber bei **npm** mit **ignore-scripts**:
- Deaktiviert auch die *eigenen* Scripts
- Malware braucht **postinstall** nicht, um Schaden anzurichten

Demo 1: Malware mit/ohne postinstall

Gegenmaßnahmen

Code nicht während der Installation ausführen

- Keine schlechte Idee, aber bei **npm** mit **ignore-scripts**:
- Deaktiviert auch die *eigenen* Scripts
- Malware braucht **postinstall** nicht, um Schaden anzurichten

Demo 1: Malware mit/ohne **postinstall**

- Es gibt ein Schlupfloch (trotz **ignore-scripts**) Code direkt während der Installation ausführen zu können
 - ab **npm 11.9.0**: mit **allow-git=none** deaktivierbar
 - wird erst mit **npm 12** standardmäßig deaktiviert sein (weil breaking change)

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

- Hätte bei `event-stream` oder `node-ipc` nicht geholfen

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

- Hätte bei `event-stream` oder `node-ipc` nicht geholfen
- Firmen nutzen oft (sinnvollerweise) einen lokalen Mirror. Dort kann noch Malware liegen, obwohl sie Upstream längst blockiert wurde.

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

- Hätte bei `event-stream` oder `node-ipc` nicht geholfen
- Firmen nutzen oft (sinnvollerweise) einen lokalen Mirror. Dort kann noch Malware liegen, obwohl sie Upstream längst blockiert wurde.
- LLM-basierte Scanner vs Prompt-Injection

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

- Hätte bei `event-stream` oder `node-ipc` nicht geholfen
- Firmen nutzen oft (sinnvollerweise) einen lokalen Mirror. Dort kann noch Malware liegen, obwohl sie Upstream längst blockiert wurde.
- LLM-basierte Scanner vs Prompt-Injection
- **Verzögert Security-Updates**, macht Notfälle wie bei der RCE in `react-server-dom-webpack` komplizierter

Gegenmaßnahmen

Updates kurz verzögern (**min-release-age**)

- Hätte bei `event-stream` oder `node-ipc` nicht geholfen
- Firmen nutzen oft (sinnvollerweise) einen lokalen Mirror. Dort kann noch Malware liegen, obwohl sie Upstream längst blockiert wurde.
- LLM-basierte Scanner vs Prompt-Injection
- **Verzögert Security-Updates**, macht Notfälle wie bei der RCE in `react-server-dom-webpack` komplizierter
- aber: **sehr effektiv für reine Development-Tools** (die nur mit vertrauenswürdigen Daten umgehen), z.B. Single-Page-Applications

Gegenmaßnahmen: Sandboxing

- Sandboxing: Code in einer isolierten Umgebung mit begrenzten Rechten ausführen
- Kann Angriffsfläche reduzieren und den Explosionsradius begrenzen

Technische Mittel zur Isolation

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich
- **Nutzeraccounts:** unterschiedliche Berechtigungen (hauptsächlich im Dateisystem)

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich
- **Nutzeraccounts:** unterschiedliche Berechtigungen (hauptsächlich im Dateisystem)
- **Container:** können Dateisystem- und Netzwerk-Zugriff einschränken

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich
- **Nutzeraccounts:** unterschiedliche Berechtigungen (hauptsächlich im Dateisystem)
- **Container:** können Dateisystem- und Netzwerk-Zugriff einschränken
- **Virtuelle Maschinen:** helfen gegen Kernel-Exploits

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich
- **Nutzeraccounts:** unterschiedliche Berechtigungen (hauptsächlich im Dateisystem)
- **Container:** können Dateisystem- und Netzwerk-Zugriff einschränken
- **Virtuelle Maschinen:** helfen gegen Kernel-Exploits
- **Physische Maschinen:** helfen gegen Hypervisor-Exploits und Hardware-Schwachstellen

Technische Mittel zur Isolation

- **Prozesse:** RAM-Isolation, separate Umgebungsvariablen, etc., bringt das Betriebssystem automatisch mit, zusätzlich *In-Process-Sandboxing* möglich
- **Nutzeraccounts:** unterschiedliche Berechtigungen (hauptsächlich im Dateisystem)
- **Container:** können Dateisystem- und Netzwerk-Zugriff einschränken
- **Virtuelle Maschinen:** helfen gegen Kernel-Exploits
- **Physische Maschinen:** helfen gegen Hypervisor-Exploits und Hardware-Schwachstellen

Container

- Sieht man oft in der Praxis
(z.B. Docker/Podman und Dev Containers) 👍
- Ziemlich leichtgewichtig, nahezu keine Performance-Nachteile
- Meiner Meinung nach **wichtigste Maßnahme** (Preis/Leistung)
und super Ausgangspunkt für weitere Verbesserungen

Demo 2

Entwicklungs-Sandbox mit Flatpak

Demo 2

Entwicklungs-Sandbox mit Flatpak

Home

Installed

Updates (Fetching...)

Settings

About

All Applications

Accessibility

Development

Education

Games

Graphics

Internet

Multimedia

Office

Science Math

System

Utilities

Application Addons

Fonts

Plasma Addons

IntelliJ IDEA Open

JetBrains s.r.o.

★★★★☆ 33 ratings

Version: 2026.1.1

Size: 2,4 GiB

License: [Apache-2.0](#)

Ages: 13+

Leading Java and Kotlin editor

****This is a community package of IntelliJ IDEA and not officially supported by JetBrains s.r.o.****

IntelliJ IDEA Open Edition is a free IDE built on open-source code that provides essential features for Java and Kotlin enthusiasts.

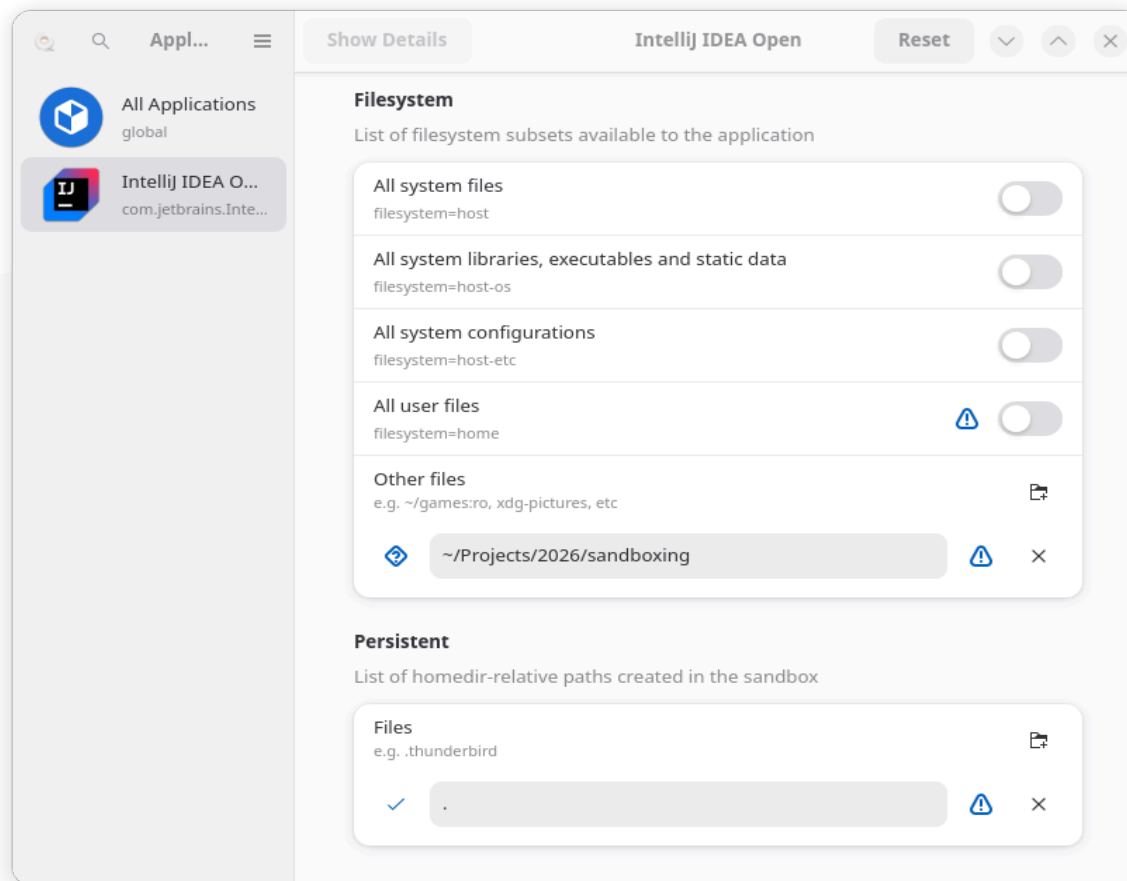
IntelliJ IDEA is an Integrated Development Environment (IDE) for professional development in Java and Kotlin. It is designed to maximize developer productivity and has a strong focus on privacy and security. It does the routine and repetitive tasks for you by providing clever code completion, static code analysis, and refactorings. It lets you focus on the bright side of software development, making it not only productive but also an enjoyable experience.

The development of modern applications involves using multiple languages, tools, frameworks, and technologies. IntelliJ IDEA is designed as an IDE for JVM languages, but numerous plugins can extend it to provide a polyglot experience.

Use IntelliJ IDEA to develop applications in the following languages that can be compiled into the JVM bytecode, namely: Java, Kotlin, Scala and Groovy

Demo 2

Entwicklungs-Sandbox mit Flatpak



Demo 3

Entwicklungs-Sandbox mit (Dev-)Containern

Demo 3

Entwicklungs-Sandbox mit (Dev-)Containern

- Dockerfile
- docker-compose.yml
- entrypoint.sh
- containers.conf

Demo 3

Entwicklungs-Sandbox mit (Dev-)Containern

Gotchas:

- Browser-Alternative: **wl-copy**
- Container: Podman in Podman braucht etwas Config
- Port-Forwarding von localhost-Sockets funktioniert nicht
- Habe keine guten Erfahrungen mit **DevContainer** + IntelliJ gemacht

Nutzeraccounts

- Separate Nutzeraccounts können gut zur Isolation benutzt werden, auf Server-Betriebssystemen seit ~1970 üblich
- Auf Desktop-Betriebssystemen leider oft hakelig
- Selbstverständlich: nicht als **root** arbeiten
 - bei der täglichen Arbeit kein **root** brauchen
 - Stolperfalle: i.d.R. ist Zugriff auf Docker-Daemon = **root**

Nutzeraccounts

- Separate Nutzeraccounts können gut zur Isolation benutzt werden, auf Server-Betriebssystemen seit ~1970 üblich
- Auf Desktop-Betriebssystemen leider oft hakelig
- Selbstverständlich: nicht als **root** arbeiten
 - bei der täglichen Arbeit kein **root** brauchen
 - Stolperfalle: i.d.R. ist Zugriff auf Docker-Daemon = **root**
 - **Demo 4: Podman hat bessere Defaults**

Nutzeraccounts

- Separate Nutzeraccounts können gut zur Isolation benutzt werden, auf Server-Betriebssystemen seit ~1970 üblich
- Auf Desktop-Betriebssystemen leider oft hakelig
- Selbstverständlich: nicht als **root** arbeiten
 - bei der täglichen Arbeit kein **root** brauchen
 - Stolperfalle: i.d.R. ist Zugriff auf Docker-Daemon = **root**
 - **Demo 4: Podman hat bessere Defaults**
- Projekte von *unterschiedlichen* Kunden sollten IMO *mindestens* durch separate User-Accounts (ohne alltäglichen **root**-Zugriff) isoliert werden

Prozesse

In-Process-Sandboxes

Prozesse

In-Process-Sandboxes

- Nutzt ihr alle täglich und sind super-praktisch, z.B. JavaScript-Runtimes oder WebAssembly
- Maschinencode wird nicht *direkt* ausgeführt
- Code wird interpretiert/generiert/verifiziert und kann I/O nur über explizit gesicherte APIs durchführen
- Deno und Node.js haben eine integrierte Sandbox, womit man weniger vertrauenswürdiges JavaScript erstmal (mehr oder weniger) geschützt starten kann

Prozesse

In-Process-Sandboxes

- **Demo 5: Sandbox von node und deno**

```
Error: Access to this API has been restricted. Use --allow-fs-read to manage permissions.  
  at open (node:internal/fs/promises:641:13)  
  at Object.readFile (node:internal/fs/promises:1287:20)  
  at file:///home/michael/Projects/2026/sandboxing/demos/5/test.js:4:29  
  at ModuleJob.run (node:internal/modules/esm/module_job:437:25)  
  at async node:internal/modules/esm/loader:639:26  
  at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:101:5) {  
  code: 'ERR_ACCESS_DENIED',  
  permission: 'FileSystemRead',  
  resource: '/home/michael/.ssh/id_ed25519'  
}
```

Prozesse

In-Process-Sandboxes

- Sich beim Entwickeln damit alleine mit In-Process-Sandboxing vor Malware zu schützen halte ich für unrealistisch (zu viele unterschiedliche Prozesse, oft nativer Code Teil von Tests/Builds)
- Oft *scheitern* Versuche, eine sichere Sandbox im Prozess zu bauen, z.B.: Java Applets (1, 2, 3), Angular.js expression sandbox, Google Caja, ActiveX, Python **rexec**, Ruby **\$SAFE**, ...

Fazit

Empfehlenswerte Supply-Chain-Security-Maßnahmen

- Wissen, welcher externer Code benutzt wird und sicherstellen, dass nicht ohne Änderung neuer Code geladen wird
- Abhängigkeiten, die nur *während der Entwicklung* verwendet werden und nur *vertrauenswürdigen Daten* verarbeiten: lieber zu alt als zu neu
- Kundenprojekte abtrennen: separate Maschinen, Nutzer-Accounts, VMs, oder (Dev-)Container
- Insbesondere sollten lokale Entwicklungstools nicht auf das Browser-Profil oder den Secret-Store zugreifen dürfen

Erhöhter Sicherheitsbedarf

- Umgang mit hochsensiblen Daten
- Login und Zahlungsabwicklung
- Security-Software
- Agentic AI
- KRITIS
- ...

Erhöhter Sicherheitsbedarf

- Umgang mit hochsensiblen Daten
- Login und Zahlungsabwicklung
- Security-Software
- Agentic AI
- KRITIS
- ...
- Konsequenzen: Security-orientiertes Design nötig, oft Performance-Einbußen, bestimmte Sprachfeatures und viele Libraries nicht (direkt) einsetzbar

Weniger Abhängigkeiten

- Weniger Komplexität und Angriffsvektoren sind mehr
- Sicherheitsprodukte sind oft selbst ein Angriffsvektor
- Neue Sprachfeatures helfen Abhängigkeiten zu vermeiden
- z.B. [Knip](#) und [Renovate](#) können helfen

Abhängigkeiten sichten

- Radikale Idee: Code nicht ungesehen ausführen
- Prior Art: Renovate und  Multiocular:

chore(deps): update dependency parse5 to v8.0.1 #53

Bearbeiten

 Zusammengeführt michael hat 1 Commits von `renovate/parse5-8.x` nach `main` letzte Woche zusammengeführt

 Diskussion 0

 Commits 1

 Geänderte Dateien 2

+14 -7 



renovate hat vor 2 Wochen kommentiert

Mitarbeiter



This PR contains the following updates:

Package	Change	Age	Confidence
parse5 (source)	8.0.0 → 8.0.1	 13d	 high

Release Notes

► inikulin/parse5 (parse5)

☐ If you want to rebase/retry this PR, check this box

Reviewer

Keine Reviewer

Label

Kein Label

Meilenstein

Kein Meilenstein

Zuständig

Niemand zuständig

2 Beteiligte



Abhängigkeiten sichten

- Radikale Idee: Code nicht ungesehen ausführen
- Prior Art: Renovate und  Multiocular:

chore(deps): update dependency parse5 to v8.0.1 #53

Bearbeiten

 Zusammengeführt michael hat 1 Commits von `renovate/parse5-8.x` nach `main` letzte Woche zusammengeführt

 Diskussion 0

 Commits 1

 Geänderte Dateien 2

+14 -7 



renovate hat vor 2 Wochen kommentiert

Mitarbeiter



This PR contains the following updates:

Package	Change	Age	Confidence
parse5 (source)	8.0.0 → 8.0.1	 13d	 high

Release Notes

► inikulin/parse5 (parse5)

☐ If you want to rebase/retry this PR, check this box

Reviewer

Keine Reviewer

Label

Kein Label

Meilenstein

Kein Meilenstein

Zuständig

Niemand zuständig

2 Beteiligte



- Ich will das aber vollständiger und besser (baue gerade etwas...)

Netzwerk-Isolation/Filterung

- Malware braucht meistens Netzwerkzugriff
- Lässt sich prinzipiell ähnlich einschränken wie Dateizugriff
- Ist praktisch aber viel schwieriger
- DNS filtern? IP-blocken? 3rd-party Software?

Netzwerk-Isolation/Filterung

- Malware braucht meistens Netzwerkzugriff
- Lässt sich prinzipiell ähnlich einschränken wie Dateizugriff
- Ist praktisch aber viel schwieriger
- DNS filtern? IP-blocken? 3rd-party Software?
- Praktisch: Anthropic hat ein paar Sandboxing-Feature/Projekte und tut etwas

Netzwerk-Isolation/Filterung

- Malware braucht meistens Netzwerkzugriff
- Lässt sich prinzipiell ähnlich einschränken wie Dateizugriff
- Ist praktisch aber viel schwieriger
- DNS filtern? IP-blocken? 3rd-party Software?
- Praktisch: Anthropic hat ein paar Sandboxing-Feature/Projekte und tut etwas
- Unpraktisch: [init-firewall.sh](#) lässt einfach alle IPv6-Anfragen durch ([issue #24952](#), "Closing for now — inactive for too long" 🤪)

Netzwerk-Isolation/Filterung

- Empfehlung: Proxy-Server in **docker-compose.yml**
- z.B. **Squid** oder **mitmproxy**

The screenshot shows the mitmproxy web interface in a browser window. The address bar indicates the URL `localhost:8081/#/flows/759b81e5-1a44-448f-9a1a-7dedadebbfe8/respons`. The interface includes tabs for **mitmproxy**, **Start**, **Options**, and **Flow**. Below the tabs are icons for **Replay**, **Duplicate**, **Revert**, **Delete**, **Download**, **Resume**, and **Abort**. A table lists intercepted flows with columns for Path, Method, Status, Size, and Time. The right panel shows the details of the selected response, which is an **HTTP/1.1 204 No Content** response from `gws`.

Path	Method	Status	Size	Time
<code>https://www.google.co...</code>	GET	302	355b	637ms
<code>http://google.com/</code>	GET	302	258b	33ms
<code>https://www.google.de/...</code>	GET	200	64.3kb	218ms
<code>https://www.google.de/i...</code>	GET	200	2.0kb	441ms
<code>https://www.google.de/...</code>	GET	200	136.9kb	242ms
<code>https://lh3.googleus...</code>	GET	200	1.2kb	82ms
<code>https://clients5.goog...</code>	GET	200	132b	71ms

Intercept: ~c 200 anticache

Request Response Details

HTTP/1.1 204 No Content

Content-Type: text/html; charset=UTF-8

Date: Tue, 20 Dec 2016 15:57:48 GMT

Server: gws

Content-Length: 0

X-XSS-Protection: 1; mode=block

X-Frame-Options: SAMEORIGIN

Bessere Isolation als Container

- Container helfen gegen Fehlbedienung/Bugs
- Container helfen oft gegen wenig aggressive Malware...
- ...sind aber machtlos bei fortgeschrittenen Attacken
- Mögliche Gegenmaßnahmen:
 - Kernel aktuell halten
 - Virtuelle Maschinen
 - **gVisor**

Standard-Library festigen

- Viele Sprachen ermöglichen es, das Verhalten der Standard-Library vom Applikationscode dauerhaft und unerwartet zu verändern
- JavaScript:
 - Schadcode kann z.B. `Array.prototype.map` überschreiben
 - Node.js: `--frozen-intrinsics`
 - Browser: bräuchte wahrscheinlich native Unterstützung
- Java:
 - `2 + 3` kann auch mal `6` sein (mit `Reflection` und `Boxed Integers`)
 - `SecurityManager` kann `Reflection` einschränken

Dynamische Code-Erzeugung verbieten

- Malware nutzt das gerne, um Code zu verschleiern
- Node: `--disallow-code-generation-from-strings`
- Browser: **Content-Security-Policy** nutzen und kein `unsafe-eval` erlauben
- Java: Ahead of Time Compilation
- Wenig Szenarien, wo man `eval` bräuchte, meist ist auch ein Build-Schritt möglich

Isolation innerhalb der Anwendung

- Kombination mehrerer Berechtigungen erzeugt oft erst den Sicherheitsvorfall (z.B. Dateisystem & Netzwerk)
- Anwendungsarchitektur anpassen (z.B. mehrere Prozesse/Services nutzen oder **wasm** statt nativem Code)
- Oder Policy-Mechanismen in die Anwendung integrieren, z.B. Config einer eigenen PoC-Implementierung für Node.js:

```
{  
  "lodash": { "node:utils": ["types"] },  
  "chalk": { "node:process": true, "node:os": true, "node:tty": true }  
}
```

- Tools wie [LavaMoat](#) peilen eine ähnliche Isolation an



FIN

Repo mit PDF und den Beispielen/Demos

Vielen Dank, insbesondere auch an:

- Stefan Kranich fürs [claude-code-devcontainer](#) Repo
- Sebastian Frühling für die [CoP-Security](#)
- Florian Ulrich für die Devcontainer-Demo
- Ludwig Richter für die Container-Netzwerkisolierung
- Jana Prechelt für den gemeinsamen Vortrag
- Jonatan Dietz fürs [Podman in Podman](#) weiterdenken

